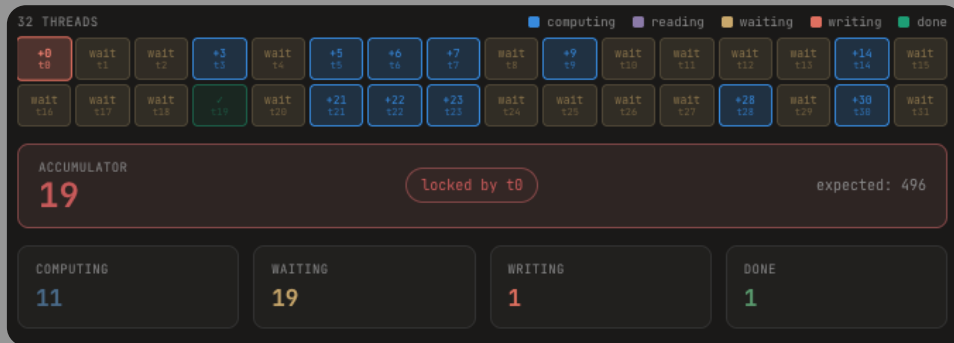


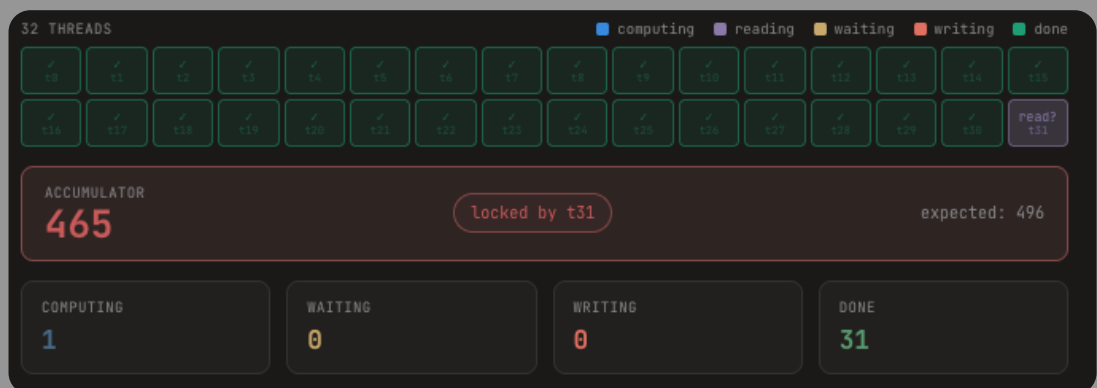
→ The Problem

⇒ atomicAdd Queue

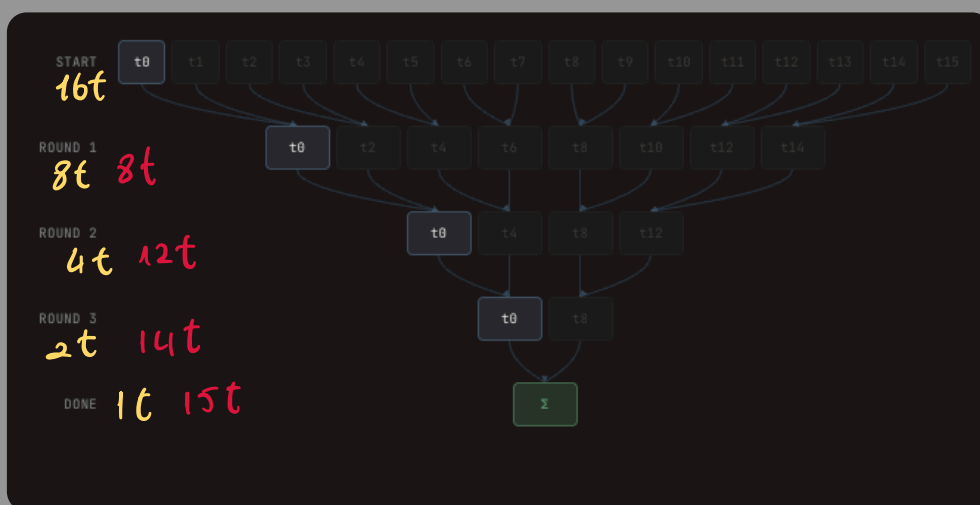


> atomicAdd serializes every thread thru one address

N threads means
N sequential writes,
which defeats the purpose of
CPU parallelism.



⇒ Naive Tree



> pay for parallelism
with idle lanes;
every sync leaves more
threads behind than
the round before

→ The fixes

⇒ Stage in SRAM

> Load once into shmem,
then reduce on-chip.

$\sim 100\times$ lower latency than DRAM,
and global-memory traffic drops from
 $O(N \log N)$ to $O(N+1)$

$\Rightarrow$ Pack Threads

> Strided

$sdata[tid] += sdata[tid + s]$
for $tid \% 2s = 0$

Starts with stride 1 and doubles it each round, pairing threads that are 2-s apart. The active threads stay scattered one-per-pair across the whole block.

> packed

$sdata[tid] += sdata[tid + s]$
for $tid < s$

Folds the array in half each round: the lowest s threads add the element s slots ahead. The active threads stay packed together at the low indices.

> strided keeps one active lane
in every warp
so the whole warp must execute.
Packed drains whole warps completely
so the scheduler can skip 'em.

$\Rightarrow$ Avoid Bank Conflicts

Strided

2-way conflict

```
idx = 2*tid; sdata[idx] += sdata[idx + 1]
```

The tree reduction written to avoid warp divergence: active threads stay contiguous, but thread t works on element 2-s-t, so neighbors are a power of two apart in memory. That stride maps several threads onto the same bank, and shared memory replays each conflicted bank one thread at a time: parallel work forced to run serially.

BANK 0 t0 t4	BANK 1	BANK 2 t1 t5	BANK 3
BANK 4 t2 t6	BANK 5	BANK 6 t3 t7	BANK 7

BANK CONFLICTS
4

Sequential

no conflict

```
sdata[tid] += sdata[tid + 8] // tid < 8
```

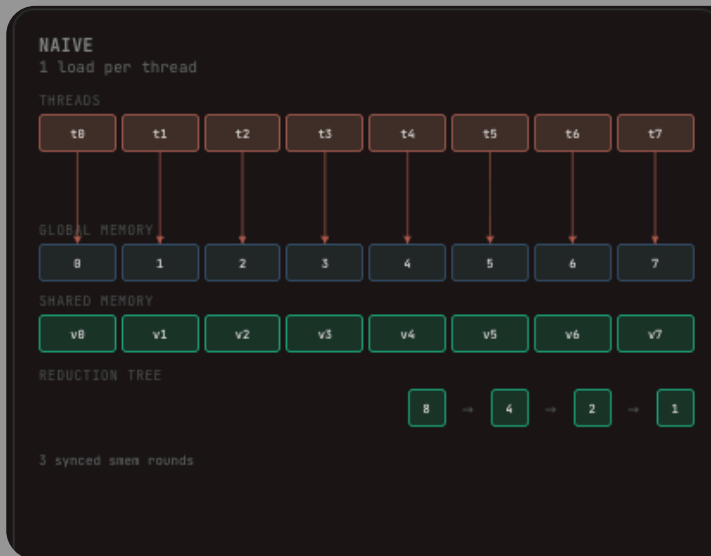
The fix: thread t simply works on element t and adds the partner s slots ahead. The active threads stay packed at the low indices, so consecutive threads touch consecutive banks, and every request is served in one cycle. Full shared-memory bandwidth, no replays.

BANK 0 t0	BANK 1 t1	BANK 2 t2	BANK 3 t3
BANK 4 t4	BANK 5 t5	BANK 6 t6	BANK 7 t7

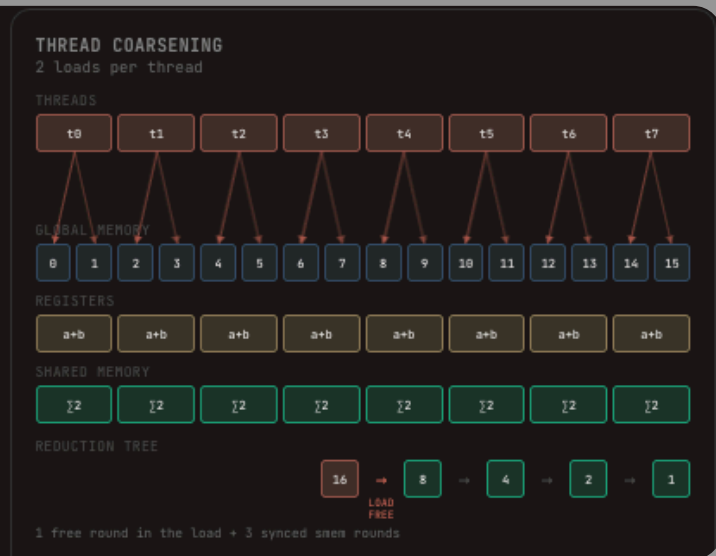
BANK CONFLICTS
0

> strided addressing maps two threads to the same bank every round.
Avoiding bank conflicts spreads them across unique banks, meaning no serialization

⇒ Thread Coarsening → physical growth.



> 1 element per thread
8 threads 8 load values
shmem holds 8 raw values
tree needs 3 rounds



> 2 elements per thread
8 thread load 16 values,
sum pairs in registers
shmem holds 8 pre-summed values
Same 3 round-tree, but each
value already represents 2 elements

→ work done in registers cost nothing!

no barrier, no sync
Coarsening moves the first reduction round
from shmem into the load itself.